

CSC 258: Algorithms for Genomic Data

Syllabus

- [Download Syllabus \(PDF\)](#)

Course Information

- CSC 258: Algorithms for Genomic Data
- Smith College Department of Computer Science
- Instructor: Halie Rando
- Semester: Fall 2026
- Class time: [TBD]
- Location: [TBD]
- Office hours: [TBD]

Course Description

This course introduces core algorithms used in bioinformatics, focusing on how biological sequences are transformed into computational representations for large-scale analysis. Students study string algorithms for alignment, k-mer counting, indexing, and genome assembly, emphasizing implementation, efficiency, and design tradeoffs. The course also explores how sequence-derived features support downstream analyses such as dimensionality reduction and association studies. Programming assignments and a final project provide hands-on experience working with real genomic data.

Prerequisites

- CSC 210: Data Structures
- Familiarity with basic programming in Python (or equivalent)
- No prior biology coursework is required

Course Overview and Goals

Biological sequence data is one of the largest and most computationally demanding forms of modern scientific data. This course examines how genomes are represented, searched, indexed, and analyzed using algorithmic techniques.

The first half of the course focuses on string-based algorithms and data structures used in genome analysis. Topics may include:

- Sequence alignment
- k-mer-based methods (as needed for analysis)
- Genome assembly (conceptual)
- Indexing methods such as suffix arrays and the Burrows–Wheeler Transform

The second half of the course examines how these representations are used in downstream analyses, including:

- Feature construction from sequence data
- Dimensionality reduction (e.g., PCA)
- Association studies (e.g., GWAS)
- Interpretation and limitations of computational representations

Throughout the course, emphasis is placed on implementation, empirical evaluation, and computational tradeoffs with an emphasis on empirical evaluation and practical implementation.

Learning Goals

By the end of the course, students will be able to:

- Explain how biological sequences are represented computationally and why representation choices matter.
- Implement core string-based algorithms used in bioinformatics.
- Implement and evaluate data structures for efficient search and counting in large-scale genomic datasets.
- Analyze tradeoffs between time, memory, and scalability in genomic data processing.
- Construct and interpret feature representations derived from sequence data.
- Critically evaluate downstream analyses that rely on computational representations.

Textbook

We will use the textbook *Bioinformatics Algorithms: An Active Learning Approach* by Phillip Compeau and Pavel Pevzner. This text emphasizes algorithmic thinking through interactive problem solving and provides clear introductions to core topics such as string alignment, genome assembly, graph algorithms, and indexing.

The book is structured around active exercises and computational challenges that reinforce implementation and design tradeoffs. We will emphasize implementation and computational intuition over formal derivations. Assigned readings will be supplemented with lecture notes and programming assignments tailored to the course.

Grading

- Weekly Programming Assignments and Problem Sets: 40%
- Midterm Assessment: 20%
- Final Project (Implementation and Benchmarking): 25%
- Final Project Write-up: 10%
- Participation and Engagement: 5%

Assignments will involve a mix of implementation, short analytical questions, and reflection on computational tradeoffs. Programming assignments will primarily use Python, though students may use other languages for the final project with instructor approval.

Final Project

Students will design and implement a substantial genomic data processing tool. The project will emphasize algorithm design, correctness, performance evaluation, and benchmarking. Students will analyze scalability and tradeoffs using real or simulated genomic datasets. A written report will summarize implementation decisions, empirical results, and interpretation.

Tentative Schedule (13 Weeks)

Unit I: Biological Sequences as Strings

- Week 1: Overview of computational genomics; DNA as data
- Week 2: Sequence alignment; dynamic programming
- Week 3: Backtracking and alignment implementation

Unit II: Counting and Assembly

- Week 4: Next-generation sequencing; reads and errors
- Week 5: k-mers and genome assembly concepts
- Week 6: Efficient counting and data structures

Unit III: Indexing and Search

- Week 7: Suffix arrays and indexing concepts
- Week 8: Burrows–Wheeler Transform and fast search
- Week 9: Alignment systems and scalability
- Midterm Assessment

Unit IV: From Sequences to Features

- Week 10: Feature representations from sequence data
- Week 11: Dimensionality reduction (PCA in genomics)

Unit V: Association and Interpretation

- Week 12: Association studies (GWAS) and interpretation
- Week 13: Final project work and presentations

Academically Engaged Time

Students should expect to spend approximately 8–10 hours per week outside of scheduled class meetings. Weekly programming assignments are designed to require sustained engagement (typically 5–7 hours), with additional time for reading and short analytical exercises. This includes time spent on programming assignments, readings, testing and debugging code, exam preparation, and final project work. Weeks with major programming assignments or project milestones may require additional time.

Collaboration and Academic Integrity (Summary)

Collaboration is encouraged at the level of discussion and idea exchange. All submitted work must be the student's own and must appropriately cite collaborators and external resources in accordance with the Smith College Honor Code.

Communication & Response Norms

The course website is the source of truth for policies, deadlines, and updates. Please use the channels below to keep questions organized and response times predictable.

Primary channels

- GitHub Issues (or PRs) are preferred for assignment questions and clarifications.
- Slack is for general course questions, peer help, and TA support. I do not monitor Slack continuously.
- Email is for private or time-sensitive matters.
- Moodle may be used occasionally for submissions or feedback, but Gradescope is the default for submissions.

Slack expectations

- Slack is asynchronous, not real-time chat.
- Slack is primarily for students to communicate with and support each other, though the teaching staff tries to check the public channels at least once per day.
- Responses may take time, especially outside business hours.
- I do not monitor Slack continuously or outside business hours.
- Slack DMs are strongly discouraged; use public channels instead.
- Office hour changes will be posted in the **#announcements** channel.
- Feel free to post playfully relevant content in **#random**.

Email expectations

- Email is checked once per business day.

- You can expect a response within 24–48 business hours.
- Emails sent outside business hours are treated as arriving the next business day.
- Emails about CSC258 should use the subject line format **CSC258: <brief description>**.

How to ask for help effectively

- Put your full question in one message.
- Include context (assignment name, deadline, and what you tried).
- Avoid sending multiple follow-ups before receiving a reply.
- Emails that do not use the required subject format and template may receive delayed responses or be redirected to office hours.

Requests that require advance notice

Some requests (extensions, recommendation letters, special arrangements) require advance notice and may have additional requirements. Please consult the course website for details.

Office hour cancellations

If office hours are cancelled, students may attend any remaining office hours that week or use email to ask brief questions. No student will be penalized for missing office hours due to a cancellation.

Assessment

There are five forms of assessment in this course:

- Weekly programming assignments
- Problem sets (short analytical or reflection questions)
- Midterm assessment
- Final project (implementation and benchmarking)
- Participation and engagement

Homework

Weekly programming assignments will emphasize algorithm implementation, correctness, and benchmarking where appropriate. Deadlines and submission links will be posted on the course website.

Execution requirement Code must run in the autograder environment to be graded. Submissions that fail to run or produce fatal errors may receive a zero or be returned ungraded, and the autograder’s execution result is authoritative.

Regrades Regrade requests must include a brief explanation of the grading concern and any relevant evidence. Regrades are handled after grades are returned and may result in a score going up, down, or remaining the same.

Extension Policy 72-hour extensions are automatically approved when submitted ahead of the deadline at the extension request form (link posted on the assignments page).

Please note:

- Up to 72 hours is granted automatically through the form.
- Extensions set a clear, final submission date.
- The extension deadline functions exactly like the original deadline; submissions after that time cannot be accepted.
- The only exception is documented extenuating circumstances communicated by the Class Deans' Office. In rare cases of serious, documented circumstances, the Class Deans' Office may contact the instructor to coordinate next steps.
- If you need more than 72 hours due to an extreme circumstance, email before the original deadline.
- Students are strongly encouraged to submit what they have early; auto-grader feedback often helps with debugging.

Attendance and Participation

Class attendance is required unless you have let the professor know about your absence (e.g., for illness, conference travel, etc.) via email. In-class activities are designed to be completed during the scheduled class time and may be difficult or impossible to replicate independently. Students who are present and participate in class receive full credit for the work. If you have an unexcused absence, you are responsible for making up any missed activities within one week, by coordinating with the instructor or TAs, preferably before the next class.

You can take notes however you want while information is being presented. During some exercises, you will be asked to “clamshell” (close) your computer to work hands-on or to sketch out ideas. If you need accommodations, please reach out to the instructor or to the Accessibility Resources Center (ARC).

Collaboration and Academic Integrity (Detailed)

Collaboration during in-class work may involve shared discussion; graded homework assignments must be completed individually. You may discuss approaches, algorithms, and design decisions with classmates, but you must write, debug, and submit your own code. You may not view another student's code before submitting your own work. Students are strongly encouraged to form study groups and to collaborate in solving the assignments. You should include a list of all students you worked with in your README file for each assignment.

Please ensure that all work you submit is ultimately the product of your own understanding rather than anyone else's.

Peer support norms

In this course, programming assignments are designed to help you build durable, individual understanding. Collaboration is encouraged at the level of ideas and design, and we strongly encourage pre-coding peer review (before you start writing code). Writing code is an individual activity so that you can accurately assess your own understanding before assessments.

Students may:

- Explain their design to a peer.
- Walk through invariants.
- Talk through edge cases.

Students may not:

- Share code.
- Screenshare code.
- Paste snippets.

You may consult online or print references on all assignments and labs. Standard language references showing syntax, usage, library docs, etc. need not be cited. All other resources must be cited as described below.

The following information is required for all submitted work:

1. The names of all collaborating students listed at the top of the submission (in your README file). We invite you to give “kudos” to students who were particularly helpful in your learning.
2. A “References” section, with in-line citations to any external resources you used. Citations should include page numbers (if a printed resource) or a direct URL (if an online resource). If you did not use any resources in completing the assignment, please state: “I did not utilize any external resources in completing this assignment.” If you include a fragment of code from any source, you should also credit that source with a comment directly in the code.
3. If you use any AI programming assistants, you should also cite this at the top of your README file, along with a description of the type of assistance provided and how it intersected with your learning process. You are responsible for being able to explain and reproduce any code you submit without assistance. Citing a tool does not necessarily make its use appropriate; all submitted work must reflect your own understanding. Using AI to diagnose or fix bugs is considered code assistance and should be cited. We discourage use of AI prompting in most situations and expect you to prioritize learning over speed.

Academic integrity process

If I have concerns about academic integrity, I am required to meet with the student to share those concerns and then refer the matter to the Academic Integrity Board to manage next steps. I do not adjudicate whether or not a violation occurred. More information is available here: <https://www.smith.edu/your-campus/offices-services/dean-college/academic-integrity-board>.